

Defining a Schema with SQLAlchemy ORM

- Due No Due Date
- Points 1
- Submitting a website url

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
-

Key Vocab

- **Schema:** the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
 - **Persist:** save a schema in a database.
 - **Engine:** a Python object that translates SQL to Python and vice-versa.
 - **Session:** a Python object that uses an engine to allow us to programmatically interact with a database.
 - **Transaction:** a strategy for executing database statements such that the group succeeds or fails as a unit.
 - **Migration:** the process of moving data from one or more databases to one or more target databases.
-

Introduction

By now you are familiar with the concept of an [ORM](https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping)  (https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping), an Object-Relational Mapper. While building your own ORM for a single class is a great way to learn about how object-oriented programming languages commonly

interact with a database, imagine you had *many* more classes. Having to test and maintain custom code to build database connectivity for each project we work on would divert our attention from what we really want to be focusing on: making cool stuff.

To save themselves and others this headache, a team of developers built the [SQLAlchemy](https://www.sqlalchemy.org/)  (<https://www.sqlalchemy.org/>) Python package.

In this lesson, we'll read about how to have SQLAlchemy link our Python classes with a database table. There's code in the `lib/sqlalchemy_sandbox.py` file set up so you can follow along with the examples below. Fork and clone this lesson if you'd like to code along.

Note: You'll never write all the code for your SQLAlchemy applications in one file like we're doing here — the setup here is kept intentionally as simple as possible so you can see everything in one place. Soon, we'll cover a more realistic SQLAlchemy file structure.

As you work through this section, it's highly recommended that you also take some time to read through the [SQLAlchemy guides](https://www.sqlalchemy.org/)  (<https://www.sqlalchemy.org/>). There's a lot more that SQLAlchemy can do than we'll be able to cover, so you're sure to discover a lot of fun new things by checking out the documentation!

SQLAlchemy ORM

SQLAlchemy is a Python library, meaning we get access to many classes and methods when we install it in our environment. There are two modes in which you can use SQLAlchemy: [SQLAlchemy Core](https://docs.sqlalchemy.org/en/14/core/)  (<https://docs.sqlalchemy.org/en/14/core/>) and [SQLAlchemy ORM](https://docs.sqlalchemy.org/en/14/orm/)  (<https://docs.sqlalchemy.org/en/14/orm/>). SQLAlchemy ORM better suits our needs here, but you may encounter SQLAlchemy Core later on in your career.

We've already configured your virtual environment to include SQLAlchemy. Simply run `pipenv install` to download `sqlalchemy` and some other helpful libraries, then `pipenv shell` to enter your virtual environment.

Defining Tables via SQLAlchemy ORM

Creating tables with SQLAlchemy ORM requires classes with four key traits:

1. Inheritance from a `declarative_base` object.

2. A `__tablename__` class attribute.
3. One or more `Column` s as class attributes.
4. A `Column` specified to be the table's primary key.

Let's take a look at a class to define a `students` table:

```
# lib/sqlalchemy_sandbox.py
from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'

    id = Column(Integer(), primary_key=True)
    name = Column(String())
```

The `declarative_base` combines a container for table metadata as well as a group of methods that act as mappers between Python and our SQL database. Inheritance from `Base` , a `declarative_base` object, allows us to avoid rewriting code.

► *Is `Base` a parent or child object?*

The `__tablename__` attribute will eventually be used as the name of our SQL database table. The table's columns are identified using `Column` objects as attributes- the optional `primary_key` argument tells SQLAlchemy that `id` will be the primary key for the `students` table.

This type of class is called a **data model**, or **model**.

Persisting the Schema

We have all of the data we need to generate a database table, but it won't happen as soon as we save our module. We need to execute a series of Python statements to do **persist** our schema. You can do this from the Python shell, but we will be using a script for this exercise.

```
# lib/sqlalchemy_sandbox.py

#!/usr/bin/env python3

# imports
from sqlalchemy import create_engine

# data models

engine = create_engine('sqlite:///students.db')
Base.metadata.create_all(engine)
```

Now run `chmod +x lib/sqlalchemy_sandbox.py` to make the script executable. Run `lib/sqlalchemy_sandbox.py` from your Pipenv shell and you should see that a `students.db` has popped up with a `students` table.

Just as with using the `sqlite3` Python module on its own, we need to start by creating a connection to the database. The engine is "home base" for the database- everything on the database side and the Python side must pass through the engine for the process to count. Here, we're pointing to a local sqlite file where our tables will be created.

The `create_all()` command on the next line tells the engine that any models that were created using `Base` as a parent should be used to create tables. if you open `students.db` in VSCode, you should see that a table exists with two columns: `id` and `name` .

Improving our Script

While inclusion of the shebang is enough to tell the interpreter that a module should be interpreted as a script, this presents a conundrum for larger projects: *how does this affect `import` statements?*

As it turns out, the interpreter gets a bit confused when an `import` statement encounters a script. As written above, `Base.metadata.create_all(engine)` would be run before the code of any module that imported `sqlalchemy_sandbox.py` . There's a trick to avoid this:

```
if __name__ == '__main__':
    engine = create_engine('sqlite:///students.db')
```

```
Base.metadata.create_all(engine)
```

`__name__` is an attribute possessed by modules that is assigned at runtime. It is assigned the value `'__main__'` if the module is the main program; that is, the one that you called from the command line.

Refactoring any scripted elements of our modules to fit into this block makes sure that the script only runs when we tell it to.

Conclusion

You should know now how to define and persist a simple schema using SQLAlchemy. In the next lesson, we will explore how to create, read, update, and delete records with SQLAlchemy ORM.

Solution Code

```
# lib/sqlalchemy_sandbox.py

from sqlalchemy import Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class Student(Base):
    __tablename__ = 'students'

    id = Column(Integer(), primary_key=True)
    name = Column(String())

if __name__ == '__main__':
```

```
engine = create_engine('sqlite:///students.db')  
Base.metadata.create_all(engine)
```

Resources

- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/) ➞ (https://docs.sqlalchemy.org/en/14/orm/)
- [6. Defining Schema with SQLAlchemy ORM - O'Reilly](https://learning.oreilly.com/library/view/essential-sqlalchemy-2nd/9781491916544/ch06.html) ➞ (https://learning.oreilly.com/library/view/essential-sqlalchemy-2nd/9781491916544/ch06.html)
- [Object-relational mapping - Wikipedia](https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping) ➞ (https://en.wikipedia.org/wiki/Object%E2%80%93relational_mapping)